

Trabalho Prático

Orientação a Objetos - FGA0158

Guilherme Lorenzi Ventura - 241011116
Gustavo Rodrigues de Noronha - 241011930
Pedro Augusto Moretti Moreira - 241011546
Thiago Alencar de Oliveira - 241011635

Julho de 2025

Sumário

1	Introdução	3
2	Diagrama de Classes	3
3	Herança	5
3.1	Hierarquia de Usuários	5
3.2	Hierarquia de Espaços	7
4	Polimorfismo	8
4.1	1. Polimorfismo por sobrescrita (dinâmico)	8
4.2	2. Polimorfismo paramétrico (uso de genéricos)	9
4.3	3. Polimorfismo por sobrecarga de métodos (estático)	10
5	Associações	10
6	Tratamento de Exceções	11
6.1	Motivação para o uso de exceções personalizadas	11
6.2	Exceções implementadas no projeto	12

1 Introdução

O presente relatório tem como objetivo apresentar a documentação do desenvolvimento do trabalho prático da disciplina de Orientação por Objetos (OO). O projeto visa aplicar os principais conceitos da programação orientada a objetos, tais como encapsulamento, herança, polimorfismo e tratamento de exceções, na construção de um sistema funcional, modular e reutilizável.

O sistema proposto busca permitir que diferentes perfis de usuários — como professores, servidores técnicos e alunos — possam realizar reservas de salas, laboratórios e demais ambientes acadêmicos, respeitando regras específicas de uso, permissões e prioridades. Dessa forma, pretende-se garantir uma gestão eficiente, transparente e automatizada da ocupação dos espaços, otimizando os recursos institucionais e aprimorando a experiência dos usuários.

Este relatório está organizado em seções que contemplam: levantamento de requisitos funcionais e não funcionais, modelagem por meio de diagramas UML, descrição das classes e seus relacionamentos, exemplos de uso do sistema, tratamento de exceções, testes realizados, além de considerações sobre as decisões de projeto, dificuldades enfrentadas e propostas de melhorias futuras.

2 Diagrama de Classes

O diagrama de classes a seguir representa a arquitetura do sistema de reserva de espaços físicos da universidade, estruturado em diferentes pacotes para refletir a separação de responsabilidades e a aplicação dos principais conceitos de orientação a objetos: herança, encapsulamento, polimorfismo e modularidade.

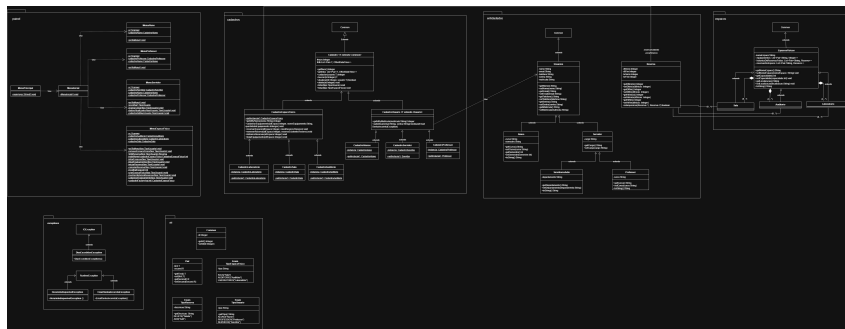


Figura 1: Diagrama de Classes

Versão online: <https://app.diagrams.net/...>

Pacote entidades: concentra as classes que representam os usuários do sistema. A superclasse **Usuario** encapsula os atributos e comportamentos comuns a todos os perfis (nome, email, telefone, senha, entre outros), e é estendida pelas subclasses especializadas:

- **Aluno:** adiciona informações específicas como matrícula, curso e semestre;
- **Servidor:** define o campo **cargo**, e é herdada por:
 - **ServidorAdm**, que adiciona o departamento e função administrativa;

- **Professor**, que informa o curso em que ministra aulas e o cargo acadêmico.

Pacote espaços: contém a hierarquia dos espaços físicos reserváveis. A classe abstrata `EspacoFisico` define os atributos comuns (nome, capacidade, localização, tipo e equipamentos). A partir dela, são especializadas as subclasses:

- `Sala`
- `Laboratorio`
- `Auditorio`

Essas subclasses permitem uma tipagem explícita para facilitar filtros, regras e relatórios de uso.

Classe Reserva: atua como entidade de ligação entre usuários e espaços físicos. Cada objeto `Reserva` associa um `Usuario` e um `EspacoFisico`, com informações como data, horário e período. Essa estrutura garante rastreabilidade das reservas e permite a geração de relatórios e históricos.

Pacote cadastros: reúne as classes responsáveis pela lógica de cadastro. A super-classe `Cadastro` é uma classe genérica e abstrata parametrizada com generics (`Cadastro<T>`), promovendo reutilização de código. A partir dela derivam os cadastros específicos:

- `CadastroAluno`, `CadastroProfessor`, `CadastroServidor` (usuários)
- `CadastroSala`, `CadastroLaboratorio`, `CadastroAuditorio` (espaços)

Além disso, a classe `CadastroEspacoFisico` agrega funcionalidades comuns aos diferentes tipos de espaços.

Pacote exceptions: contém as exceções personalizadas utilizadas no sistema. As principais são:

- `HorarioIndisponivelException`: lançada ao tentar reservar um espaço já ocupado;
- `DiasExcedidosException`: impede que alunos reservem por mais de um dia;
- `HorarioNaoPermitidoAluno`: usada quando um aluno tenta reservar fora do horário permitido;
- `EmailSenhaIncorretaException`: trata falhas de autenticação.

Essas exceções estendem a classe `RuntimeException`, via uma hierarquia intermediária definida pelo projeto.

Pacote painel: responsável pela interface do sistema com o usuário, especialmente via terminal. Os menus `MenuAluno`, `MenuProfessor` e `MenuServidor` oferecem opções específicas de acordo com o perfil, seguindo o princípio de separação de responsabilidades.

Resumo estrutural: o sistema apresenta uma arquitetura extensível, bem modularizada e fortemente tipada. Relações de herança modelam especializações de usuários e espaços. Associações explícitas ligam reservas aos seus responsáveis e locais. O uso de generics, encapsulamento e exceções personalizadas reforça a robustez do projeto, além de facilitar sua manutenção futura.

3 Herança

A herança é um dos pilares da programação orientada a objetos e permite a criação de uma *hierarquia de classes*, na qual classes mais especializadas (subclasses) herdam atributos e comportamentos de classes mais genéricas (superclasses). Isso favorece o *reaproveitamento de código*, reduz duplicações e facilita a manutenção e expansão do sistema.

No sistema de reservas desenvolvido, a herança foi essencial para representar **dois conjuntos hierárquicos principais**: os diferentes tipos de usuários e os diferentes tipos de espaços físicos da universidade.

3.1 Hierarquia de Usuários

A classe `Usuario` foi criada como uma superclasse abstrata que representa qualquer pessoa que possa realizar reservas no sistema. Ela contém os atributos básicos compartilhados entre todos os usuários — como nome, email e matrícula —, servindo como base para especializações posteriores.

```
1 public abstract class Usuario {
2     protected String nome;
3     protected String email;
4     protected String matricula;
5
6     public Usuario(String nome, String email, String matricula) {
7         this.nome = nome;
8         this.email = email;
9         this.matricula = matricula;
10    }
11 }
```

Listing 1: Classe base Usuario

A partir dessa superclasse, foram criadas subclasses como `Aluno`, `Servidor`, `Professor` e `ServidorAdm`. Cada uma adiciona atributos próprios conforme o perfil do usuário, refletindo diferentes regras de uso e permissões no sistema.

Exemplo 1: A classe `Aluno` representa estudantes e, por isso, contém atributos como curso e semestre:

```
1 private String curso;
2 private String semestre;
3
4 public Aluno(String nome, String email, String telefone, String senha,
5     String curso, String matricula, String semestre) {
6     super(nome, email, telefone, senha, matricula);
7     this.curso = curso;
8     this.semestre = semestre;
9 }
```

Listing 2: Subclasse Aluno

Exemplo 2: A classe `Professor` herda de `Servidor` e adiciona o campo `curso`, que representa a área em que o professor ministra aulas:

```
1 public class Professor extends Servidor {  
2     private String curso;  
3  
4     public Professor(String nome, String email, String telefone, String  
5         senha, String matricula, String cargo, String curso) {  
6         super(nome, email, telefone, senha, matricula, cargo);  
7         this.curso = curso;  
8     }  
9 }
```

Listing 3: Subclasse Professor

Exemplo 3: A classe `Servidor` representa funcionários administrativos ou técnicos e adiciona o campo `cargo`:

```
1 public class Servidor extends Usuario{  
2     private String cargo;  
3  
4     public Servidor(String nome, String email, String telefone, String  
5         senha, String matricula, String cargo) {  
6         super(nome, email, telefone, senha, matricula);  
7         this.cargo = cargo;  
8     }  
9 }
```

Listing 4: Subclasse Servidor

Exemplo 4: A classe `ServidorAdm` estende `Servidor` e adiciona o atributo `departamento`, ao qual está vinculado o servidor:

```
1 public class ServidorAdm extends Servidor{  
2     private String departamento;  
3  
4     public ServidorAdm(String nome, String email, String telefone,  
5         String senha, String matricula, String cargo, String departamento  
6         ) {  
7         super(nome, email, telefone, senha, matricula, cargo);  
8         this.departamento = departamento;  
9     }  
10 }
```

Listing 5: Subclasse ServidorAdm

3.2 Hierarquia de Espaços

O mesmo princípio foi aplicado para os espaços físicos reserváveis. A superclasse abstrata `EspacoFisico` representa qualquer ambiente da universidade que pode ser reservado. Ela herda de uma classe utilitária chamada `Common`, responsável apenas por fornecer um identificador (`id`) comum às entidades do sistema.

```
1 public abstract class EspacoFisico extends Common{
2     private String nomeEspaco;
3     private int capacidade;
4     private String localizacao;
5     private List<Pair<String, Integer>> equipamentos = new ArrayList<>()
6     ;
7     private List<Pair<String, Reserva>> historicoDeReservasFeitas = new
8         ArrayList<>();
9     private List<Reserva> reservasDoEspaco = new ArrayList<>();
10
11     public EspacoFisico(String nomeEspaco , int capacidade, String
12         localizacao) {
13         this.nomeEspaco = nomeEspaco;
14         this.capacidade = capacidade;
15         this.localizacao = localizacao;
16     }
17
18     @Override
19     public String toString() {
20         return "Nome do Espaco: " + nomeEspaco + ", Capacidade: " +
21             capacidade + ", Localizacao: " + localizacao;
22     }
23 }
```

Listing 6: Classe base `EspacoFisico`

A partir dela, foram implementadas subclasses específicas:

- Sala: usada para aulas regulares.
- Laboratorio: equipada com recursos técnicos, como computadores.
- Auditorio: utilizado para eventos maiores e com necessidades específicas.

Cada uma dessas subclasses mantém a estrutura herdada, mas permite especializações futuras conforme o tipo de uso esperado para o espaço:

```
1 public class Sala extends EspacoFisico {
2     public Sala(String nomeEspaco, int capacidade, String loc) {
3         super(nomeEspaco, capacidade, loc);
4     }
5 }
```

Listing 7: Subclasse `Sala`

```
1 public class Laboratorio extends EspacoFisico {
2     public Laboratorio(String nomeEspaco, int capacidade, String loc) {
3         super(nomeEspaco, capacidade, loc);
4     }
5 }
```

Listing 8: Subclasse `Laboratorio`

```
1 public class Auditorio extends EspacoFisico {
2     public Auditorio(String nomeEspaco, int capacidade, String loc) {
3         super(nomeEspaco, capacidade, loc);
4     }
5 }
```

Listing 9: Subclasse Auditorio

4 Polimorfismo

O polimorfismo é um dos pilares da programação orientada a objetos e refere-se à capacidade de uma mesma interface — como o nome de um método — se comportar de maneiras diferentes dependendo do contexto. Essa característica permite maior flexibilidade, extensibilidade e reaproveitamento de código, reduzindo acoplamentos rígidos e aumentando a coesão do sistema.

No sistema de reservas, o polimorfismo foi aplicado de forma prática em três níveis distintos: **por sobrescrita (dinâmico)**, **paramétrico (genéricos)** e **por sobrecarga de métodos (estático)**. A seguir, detalhamos como cada um desses tipos foi implementado no código do projeto.

4.1 1. Polimorfismo por sobrescrita (dinâmico)

O polimorfismo dinâmico ocorre quando uma subclasse redefine um método herdado de uma superclasse. Em Java, todas as classes herdam, direta ou indiretamente, da classe `Object`, que define métodos fundamentais como `toString()`. Esse método pode ser sobrescrito para fornecer uma representação textual mais significativa do objeto.

No sistema desenvolvido, classes como `Aluno` e `Professor` sobrescrevem o método `toString()` para exibir seus dados de forma personalizada. Isso permite que uma lista de objetos de diferentes tipos — tratados por uma referência comum — seja exibida de maneira contextual, conforme o tipo real de cada instância.

```
1 @Getter
2 @Setter
3 public class Aluno extends Usuario {
4     private String curso;
5     private String semestre;
6
7     public Aluno(String nome, String email, String telefone, String
8         senha, String curso, String matricula, String semestre) {
9         super(nome, email, telefone, senha, matricula);
10        this.curso = curso;
11        this.semestre = semestre;
12    }
13
14    public Aluno() {}
15
16    @Override
17    public String toString() {
18        return "Aluno: " +
19            "nome=" + getNome() +
20            ", telefone=" + getTelefone() +
21            ", matricula=" + getMatricula() +
```



```
21         ", curso=" + curso +
22         ", semestre=" + semestre + "\n";
23     }
24 }
```

Listing 10: Classe Aluno com sobrescrita de toString()

Esse uso do método `toString()` demonstra o polimorfismo dinâmico: mesmo que a chamada seja feita por uma referência genérica (como `Usuario`), a execução será resolvida com base na classe concreta em tempo de execução.

4.2 2. Polimorfismo paramétrico (uso de genéricos)

O polimorfismo paramétrico refere-se ao uso de tipos genéricos para escrever código que funcione com múltiplos tipos de forma segura e reutilizável. Isso é amplamente empregado no projeto por meio das classes `Pair<T, R>` e `Cadastro<T>`.

```
1 public class Pair<T, R> {
2     private T first;
3     private R second;
4
5     public Pair(T first, R second) {
6         this.first = first;
7         this.second = second;
8     }
9 }
```

Listing 11: Classe Pair com parâmetros genéricos

A classe abstrata `Cadastro<T>` é parametrizada com um tipo genérico que representa a entidade manipulada (por exemplo, `Aluno`, `Professor`, `Sala`). Com isso, é possível reutilizar métodos para diferentes contextos, como cadastro de usuários e de espaços físicos.

```
1 public void listar(TipoUsuario tipo){
2     System.out.println("\n" + tipo.getTipo() + " cadastrados:");
3     for (Pair<T, OffsetDateTime> par : db){
4         System.out.println(par.getFirst().toString());
5     }
6 }
```

Listing 12: Método listar com tipo genérico T

```
1 public void listar(TipoEspacoFisico tipo){
2     System.out.println("\n" + tipo.getTipo() + "s cadastrado(a)s:");
3     for (Pair<T, OffsetDateTime> par : db){
4         System.out.println(par.getFirst().toString());
5     }
6 }
```

Listing 13: Sobrecarga do listar para espaços físicos

Esse uso de genéricos permite a generalização das operações de listagem, cadastro e validação, sem duplicação de código.

4.3 3. Polimorfismo por sobrecarga de métodos (estático)

A sobrecarga ocorre quando métodos com o mesmo nome, mas diferentes assinaturas (parâmetros), são definidos na mesma classe. Esse tipo de polimorfismo é resolvido em tempo de compilação e melhora a versatilidade da interface.

No projeto, há sobrecarga de métodos como `listar()`, na classe `Cadastro<T>`, e também sobrecarga de construtores, como no exemplo a seguir:

```
1 public Aluno(String nome, String email, String telefone, String senha,
2   String curso, String matricula, String semestre) {
3   super(nome, email, telefone, senha, matricula);
4   this.curso = curso;
5   this.semestre = semestre;
6 }
```

Listing 14: Sobrecarga de construtores em Aluno.java

```
1 public Aluno() {
2 }
```

Listing 15: Construtor vazio em Aluno.java

5 Associações

As associações representam os vínculos lógicos entre classes no sistema, possibilitando a colaboração entre objetos e a modelagem de relações reais, como "quem reserva o quê", "quem gerencia quais entidades" e "quem interage com o sistema".

No sistema de reservas desenvolvido, identificam-se as seguintes associações principais, conforme representado no diagrama de classes:

- **Usuário → Reserva**

Cada instância da classe `Reserva` está associada a um objeto da classe `Usuario`, representando o responsável pela reserva. A associação é unidirecional na implementação da classe `Reserva` (que armazena uma referência ao `Usuario`), mas conceitualmente é bidirecional, pois os espaços físicos também mantêm um histórico das reservas realizadas por cada usuário. Essa ligação permite mapear o histórico de uso de cada pessoa, bem como aplicar regras específicas por tipo de usuário (como prioridade ou restrição de horários).

- **Reserva ↔ EspacoFisico**

Cada objeto da classe `Reserva` também contém uma referência ao objeto do tipo `EspacoFisico` correspondente, estabelecendo qual sala, laboratório ou auditório foi reservado. Essa associação possibilita consultar o histórico de uso de cada espaço, verificar disponibilidade e aplicar validações como conflitos de horário ou capacidade máxima.

- **EspacoFisico ↔ Pair<Usuario, List<Reserva>**

A classe `EspacoFisico` contém internamente um par (`Pair<Usuario, List<Reserva>`) que armazena, para cada usuário, todas as reservas já realizadas naquele espaço. Essa associação implícita reforça a ligação bidirecional entre usuários e espaços físicos, permitindo ao sistema rastrear facilmente quem utilizou qual espaço e em

que horários, além de facilitar a aplicação de políticas como limite de reservas por usuário.

- **Cadastro<T> → Entidades**

As classes de cadastro (`CadastroAluno`, `CadastroProfessor`, `CadastroServidor`, etc.) armazenam listas de pares contendo uma entidade do tipo `T` (onde `T` estende `Usuario`) e um `OffsetDateTime` correspondente ao momento do cadastro. A estrutura genérica `Pair<T, OffsetDateTime>` promove flexibilidade e rastreabilidade, possibilitando ações como auditoria de cadastros e ordenação por tempo.

- **CadastroEspacoFisico → Espaços específicos**

A classe `CadastroEspacoFisico` encapsula lógica comum de manipulação de espaços (como salas, laboratórios e auditórios). Embora essas subclasses sejam especializadas, elas compartilham atributos e métodos herdados da superclasse abstrata `EspacoFisico`, e todas são gerenciadas por instâncias de `CadastroEspacoFisico`, que atua como ponto central para criação, listagem e exclusão desses objetos.

- **Menus → Cadastros e Usuários**

Os menus do sistema (`MenuAluno`, `MenuProfessor`, `MenuServidor`, `MenuEspacoFisico`) são responsáveis por interagir com o usuário via terminal. Cada menu é especializado para um tipo de usuário e contém associações explícitas com os cadastros e serviços correspondentes. Por exemplo, `MenuAluno` se comunica com `CadastroAluno` e `MenuEspacoFisico` para realizar reservas, enquanto `MenuProfessor` possui funcionalidades específicas para professores. Essa associação promove coesão e separação clara de responsabilidades entre camadas de apresentação (menus) e lógica de domínio (cadastros, entidades, reservas).

- **Enums → Espaços e Reservas**

As classes `TipoEspacoFisico` e `TipoHorario` são utilizadas para categorizar os espaços físicos e horários permitidos. Essas enums são utilizadas tanto nas instâncias de `EspacoFisico` quanto na lógica de reserva, permitindo a padronização de atributos como “tipo de ambiente” e “turno de reserva”, e auxiliando no controle das regras de negócio por perfil de usuário.

6 Tratamento de Exceções

O tratamento de exceções é um mecanismo essencial em sistemas robustos, pois permite lidar com situações anômalas ou não previstas de maneira controlada. Em vez de interromper abruptamente a execução do programa, exceções capturam comportamentos que violam regras pré-estabelecidas e oferecem respostas apropriadas, tanto para o sistema quanto para o usuário.

No sistema de reservas, o uso de **exceções personalizadas** foi crucial para garantir o cumprimento de regras de negócio específicas e para separar logicamente os fluxos normais daqueles que representam falhas operacionais ou violações de políticas institucionais.

6.1 Motivação para o uso de exceções personalizadas

Diferentes tipos de usuários possuem permissões distintas para realizar reservas, e o sistema deve aplicar restrições adequadas para garantir integridade, previsibilidade e justiça

no uso dos espaços institucionais. Para isso, foram criadas exceções específicas que representam cada tipo de violação de regra de negócio relevante. Essas exceções são utilizadas para interromper o fluxo de operações inválidas e fornecer mensagens claras aos usuários, promovendo maior usabilidade e segurança.

6.2 Exceções implementadas no projeto

1. HorárioIndisponivelException

Essa exceção é lançada quando se tenta reservar um espaço em um horário já ocupado. É essencial para evitar sobreposição de reservas, garantindo o uso exclusivo de ambientes físicos e evitando conflitos operacionais.

```
1 public class HorárioIndisponivelException extends RuntimeException {  
2     public HorárioIndisponivelException() {  
3         super("Horário indisponível para reserva.");  
4     }  
5 }
```

Listing 16: Classe HorárioIndisponivelException.java

2. DiasExcedidosException

Essa exceção modela a limitação de reservas com muita antecedência, impedindo que usuários bloqueiem horários de forma excessiva ou desnecessária. Ela contribui para a previsibilidade e equidade na alocação de espaços, restringindo abusos no agendamento.

```
1 import java.io.IOException;  
2  
3 public class DiasExcedidosException extends IOException {  
4     public DiasExcedidosException() {  
5         super("O número de dias excedeu o limite permitido para a  
6             reserva.");  
7     }  
8 }
```

Listing 17: Classe DiasExcedidosException.java

3. EmailSenhaIncorretaException

Utilizada para representar falhas no processo de autenticação, essa exceção reforça a segurança do sistema ao impedir que usuários não autorizados acessem funcionalidades com base em credenciais inválidas.

```
1 public class EmailSenhaIncorretaException extends RuntimeException {  
2     public EmailSenhaIncorretaException() {  
3         super("Email ou senha incorretos.");  
4     }  
5 }
```

Listing 18: Classe EmailSenhaIncorretaException.java